



Mockito: introdução ao Mocking

Neste módulo, conheceremos o Mockito e dar os primeiros passos no mundo do **mocking e stubbing**, técnicas que nos ajudam a isolar componentes durante os testes para garantir que cada parte do sistema funcione corretamente de maneira independente. Vamos entender quando e por que criar simulações de objetos, reforçando a importância de focar o teste na unidade que queremos validar, sem depender de fatores externos. Em seguida, veremos como realizar a **criação de mocks**, aprendendo a configurar comportamentos simulados e controlar respostas específicas para diferentes situações. Com o apoio de exemplos práticos e esquemas visuais, ficará mais claro como aplicar esses conceitos no dia a dia. Depois, exploraremos a **simulação de dependências**, analisando como o uso de mocks pode tornar nossos testes mais rápidos, previsíveis e focados, mesmo em sistemas complexos. Para fechar o módulo, estudaremos a **verificação de interações entre objetos**, compreendendo como garantir que nossas classes se comuniquem da maneira esperada, validando não só o resultado, mas também o caminho percorrido.

Conceitos de mocking e stubbing

Neste tema, vamos entrar no universo dos **conceitos de mocking e stubbing**, fundamentos essenciais para construirmos testes mais focados, isolados e realistas. À medida que os sistemas se tornam mais complexos, interagindo com bancos de dados, APIs externas ou serviços internos, precisamos de estratégias para testar unidades individuais sem depender desses elementos externos. É exatamente aí que entram os mocks e os stubs.

O conceito de mock, ou objeto simulado, refere-se à criação de uma versão controlada de uma dependência real, utilizada no lugar do componente

original durante o teste. Ao utilizarmos mocks, conseguimos isolar o comportamento da unidade que queremos testar, assegurando que qualquer falha ou variação em componentes externos não interfira nos resultados. O mock nos permite definir como a dependência deve se comportar, quais respostas ela deve retornar e como ela deve interagir com o sistema que estamos testando.

O stubbing, por sua vez, é o ato de configurar o comportamento do mock, determinando como ele deve responder a chamadas específicas. Podemos definir, por exemplo, que, quando um determinado método for chamado com certos parâmetros, ele deve retornar um valor específico. O stubbing nos dá controle total sobre as respostas das dependências simuladas, permitindo simular diferentes cenários de forma simples e segura.

No contexto dos testes, os mocks nos ajudam a verificar não apenas os resultados das operações, mas também as interações entre os componentes. Podemos checar, por exemplo, se um método foi chamado, quantas vezes ele foi invocado e com quais parâmetros. Esse tipo de verificação é essencial para garantir que o fluxo de comunicação entre as partes do sistema esteja correto e os contratos de interação sejam respeitados.

Quando pensamos em boas práticas, é importante lembrar que mocks e stubs devem ser utilizados com responsabilidade. Se abusarmos da criação de mocks ou se os configurarmos de maneira muito complexa, nossos testes podem se tornar frágeis e difíceis de manter. O ideal é utilizar mocks para isolar dependências externas, controlar o ambiente do teste e focar no comportamento da unidade de código que realmente queremos validar.

Uma ferramenta muito utilizada para trabalhar com mocks e stubs em projetos Java é o Mockito, que vamos explorar em detalhes nos próximos temas. O Mockito nos permite criar mocks de maneira simples, definir comportamentos esperados e verificar interações com poucas linhas de código, tornando o processo de escrita de testes mais fluido e produtivo.

Além de facilitar o isolamento das unidades de teste, o uso de mocks e stubs também nos permite acelerar a execução dos testes, já que não

precisamos acessar recursos externos reais, que poderiam tornar os testes mais lentos e menos previsíveis. Dessa forma, conseguimos manter a agilidade no ciclo de desenvolvimento e promover uma cultura de testes contínuos e integrados ao processo de construção do software.

Ao dominarmos os conceitos de mocking e stubbing, expandimos nossa capacidade de escrever testes mais sólidos, confiáveis e alinhados às boas práticas de engenharia de software. Construímos sistemas mais modulares, com dependências bem definidas, e tornamos o processo de desenvolvimento mais seguro e adaptável às mudanças.

Nos próximos temas, vamos colocar esses conceitos em prática, aprendendo a criar mocks, definir comportamentos simulados e verificar interações entre objetos utilizando as ferramentas que o JUnit 5 e o Mockito nos oferecem. Assim, seguiremos fortalecendo nossa habilidade de construir testes que realmente contribuem para a qualidade e a evolução dos sistemas que desenvolvemos.

Criação de mocks

Neste tema, vamos aprender sobre a **criação de mocks**, um passo fundamental para aplicarmos na prática os conceitos de isolamento de dependências que discutimos anteriormente. Criar mocks é o ponto de partida para escrever testes que focam exclusivamente na unidade de código que queremos validar, sem influências externas que comprometam a clareza ou a confiabilidade dos resultados.

No Mockito, a criação de mocks é simples e direta. Utilizamos o método estático `mock()`, fornecido pela biblioteca, para criar uma instância simulada de uma classe ou interface. Essa instância pode ser configurada para se comportar de maneiras específicas durante o teste. Quando criamos um mock, não estamos criando um objeto real da classe original. Em vez disso, o Mockito gera dinamicamente uma versão falsa do objeto, capaz de simular respostas e registrar interações.

A criação de mocks é geralmente feita no início do teste, durante a fase de preparação do cenário. Podemos criar mocks diretamente no método de

teste ou utilizar a anotação `@Mock` combinada com a anotação `@ExtendWith(MockitoExtension.class)` para inicializá-los automaticamente em nossos testes de JUnit 5. Essa segunda abordagem ajuda a manter o código de teste mais limpo e organizado, especialmente em classes que utilizam múltiplos mocks.

Uma vez criado o mock, podemos configurá-lo usando o método `when().thenReturn()`. Com esse padrão, definimos o que deve acontecer quando um método específico for chamado no mock. Por exemplo, podemos dizer que quando o método `buscarCliente()` for chamado, o mock deve retornar um cliente simulado, preparado previamente para o teste. Essa configuração nos dá total controle sobre o comportamento das dependências durante a execução dos testes.

Outro recurso útil é a possibilidade de configurar exceções. Com o método `when().thenThrow()`, podemos simular situações em que a dependência lança um erro, permitindo que testemos o tratamento adequado de exceções pelo código sob teste. Essa prática é especialmente importante para validar fluxos alternativos e garantir a robustez do sistema.

Além da configuração básica, o Mockito permite a criação de mocks mais sofisticados, como mocks parciais (`spy`), que combinam comportamentos reais e simulados. O uso de spies é interessante quando queremos testar um objeto real, mas ainda assim controlar alguns de seus métodos durante o teste.

Na prática, criar mocks corretamente significa preparar um ambiente de teste onde todas as variáveis são conhecidas e controladas. Dessa forma, conseguimos focar exclusivamente na lógica da unidade testada, sem surpresas vindas de interações com bancos de dados, serviços externos ou componentes complexos do sistema.

É importante lembrar que, ao trabalharmos com mocks, devemos evitar configurar comportamentos desnecessários ou irrelevantes para o teste em questão. O excesso de configuração pode tornar o teste mais difícil de entender e mais frágil diante de mudanças no código. O ideal é simular somente o que for realmente necessário para o cenário de teste que

estamos validando.

A criação de mocks é uma habilidade que, ao ser bem dominada, nos permite escrever testes mais rápidos, mais confiáveis e mais alinhados ao princípio de testes unitários: verificar o comportamento de pequenas unidades de código de maneira isolada.

Nos próximos temas, vamos seguir aprofundando o uso de mocks, explorando como simular dependências de maneira mais elaborada e como verificar as interações entre os objetos para garantir que o comportamento do sistema esteja correto em diferentes cenários. Assim, continuamos construindo uma base sólida para a prática profissional de testes de software.

Simulação de dependências

Neste tema, vamos aprofundar nosso estudo sobre a **simulação de dependências** nos testes. A simulação é uma etapa fundamental para garantir que nossos testes sejam verdadeiramente focados na unidade de código que desejamos validar, sem influência de fatores externos que comprometam a previsibilidade e a clareza dos resultados.

Quando simulamos dependências, estamos criando um ambiente controlado onde cada resposta, cada comportamento de um componente externo é definido previamente. Isso nos permite testar a lógica interna da unidade de código em condições específicas, sem depender da disponibilidade ou do comportamento real de sistemas externos como bancos de dados, serviços de autenticação, APIs de terceiros ou mesmo outras classes do nosso próprio projeto.

A simulação de dependências é feita, principalmente, através dos mocks que criamos com o Mockito. Após instanciarmos um mock, podemos definir seus comportamentos usando métodos como `when().thenReturn()` para respostas esperadas ou `when().thenThrow()` para simular falhas. Dessa forma, conseguimos criar diferentes cenários de teste apenas ajustando as respostas das dependências simuladas.

Por exemplo, ao testar um serviço de cadastro de usuários, podemos

simular um repositório que retorna que o e-mail já existe no sistema ou, em outro cenário, simular que o usuário pode ser cadastrado normalmente. Alterando apenas o comportamento do mock, conseguimos testar ambos os fluxos sem precisar de um banco de dados real.

A simulação também permite controlar estados e sequências de chamadas. Podemos configurar o mock para retornar valores diferentes em chamadas subsequentes ao mesmo método, utilizando o `thenReturn()` encadeado ou métodos como `thenReturnAnswer()` para respostas mais dinâmicas. Essa abordagem é útil para testes de fluxos que envolvem mudanças de estado ao longo do tempo, como filas de processamento ou transações financeiras.

Além de simular retornos, podemos também utilizar a simulação para testar o tratamento de exceções, preparando o mock para lançar erros e verificando se a unidade testada reage de maneira adequada, como capturando a exceção, registrando um log ou realizando uma operação alternativa.

Outro aspecto importante da simulação de dependências é garantir que os testes permaneçam rápidos e estáveis. Ao eliminar a necessidade de acesso a recursos externos reais, conseguimos rodar nossa suíte de testes em qualquer ambiente, sem a necessidade de configurações complexas ou dependências externas, fortalecendo a confiabilidade dos testes e facilita a integração contínua.

Devemos sempre buscar equilíbrio na simulação de dependências. Simular apenas o necessário torna nossos testes mais claros e fáceis de entender. Se exagerarmos, criando cenários extremamente artificiais ou complexos, podemos acabar testando somente a configuração dos mocks, e não o comportamento real do sistema que queremos validar.

Simular dependências corretamente nos dá mais controle, mais confiança nos resultados e mais agilidade no ciclo de desenvolvimento. Passamos a construir testes que são verdadeiramente unitários, que isolam causas e efeitos e que nos ajudam a encontrar defeitos de forma rápida e precisa.

Com esse domínio sobre a simulação de dependências, estamos prontos

para evoluir ainda mais no uso de testes com mocks. Nos próximos temas, vamos explorar como verificar as interações entre os objetos simulados e as unidades sob teste, garantindo não somente o resultado, mas também que o caminho percorrido até ele seja o correto. Assim, seguimos fortalecendo nossa habilidade de construir software com qualidade e segurança.

Verificação de interações entre objetos

Neste tema, vamos aprender sobre a verificação de interações entre objetos, um passo essencial para garantir que nossos testes validem não somente os resultados, mas também os comportamentos internos esperados durante a execução do sistema. Testar interações nos permite confirmar que os componentes estão se comunicando da maneira correta, respeitando os contratos de chamada e a lógica de negócio estabelecida.

Ao utilizar mocks com o Mockito, podemos não apenas simular respostas, mas também monitorar como os objetos simulados foram utilizados durante o teste. Essa verificação é feita através do método `verify()`, que nos permite checar se um determinado método foi chamado em um mock, quantas vezes ele foi invocado e com quais argumentos. Essa capacidade de verificação traz um nível de precisão importante para os nossos testes.

Com o `verify()`, conseguimos validar, por exemplo, se um serviço chamou corretamente um repositório para salvar dados, se uma notificação foi enviada após uma operação ser concluída ou se uma transação foi iniciada e finalizada corretamente. Esse tipo de teste é especialmente útil em sistemas orientados a eventos ou com fluxos de trabalho que envolvem múltiplas etapas e dependências.

A verificação básica envolve simplesmente confirmar que um método foi chamado, utilizando a sintaxe `verify(mock).metodoEsperado()`. Podemos também especificar a quantidade de vezes que a chamada deveria ter ocorrido, usando métodos como `times()`, `atLeastOnce()` ou `never()`. Isso nos permite validar cenários como tentativas únicas de execução, chamadas repetidas ou a ausência completa de interação em determinados fluxos.

Outro recurso interessante é a verificação de ordem de chamadas, feita

com o objeto InOrder do Mockito. Em casos onde a sequência das interações é crítica, como no processamento de etapas de um pedido ou na comunicação com sistemas externos, o InOrder nos permite garantir que os métodos foram chamados na ordem correta.

Também podemos capturar os argumentos passados para os métodos utilizando ArgumentCaptor. Esse recurso é muito útil para validar não somente se o método foi chamado, mas também se os dados transmitidos estavam corretos. Por exemplo, podemos verificar se uma entidade salva em um repositório foi criada com os atributos esperados.

Ao focarmos na verificação de interações, estamos validando comportamentos, e não apenas resultados. Isso é fundamental para garantir que o sistema esteja funcionando como desenhado, respeitando a lógica dos fluxos e as responsabilidades de cada componente.

No entanto, é importante usar a verificação de interações com critério. Testes que verificam interações excessivamente específicas podem se tornar frágeis diante de pequenas mudanças internas no código, mesmo que o comportamento externo do sistema permaneça correto. O ideal é focar nas interações que são realmente relevantes para a lógica de negócio, garantindo o equilíbrio entre testes robustos e manutenção prática.

Ao dominarmos a verificação de interações entre objetos, enriquecemos nossos testes com uma nova camada de precisão e confiança. Passamos a garantir não apenas que o sistema produz o resultado esperado, mas também que ele alcança esse resultado de maneira correta, segura e previsível.

Com essa habilidade, concluímos um ciclo importante na construção de testes com mocks. Nos próximos temas, avançaremos para testar cenários mais complexos, trabalhando com integrações e ampliando ainda mais nossa capacidade de desenvolver sistemas confiáveis e preparados para os desafios reais.

Conteúdo Bônus

Recomendamos a leitura do livro *Mockito Made Clear: Java Unit Testing with Mocks, Stubs, and Spies*, de Ken Kousen, como material complementar para este módulo. Esta obra oferece uma introdução prática ao framework Mockito, abordando desde os conceitos fundamentais de mocks, stubs e spies até técnicas avançadas de verificação de interações e uso de matchers personalizados. Com exemplos claros baseados em JUnit 5, o autor demonstra como isolar dependências externas e testar unidades de código de forma eficaz, promovendo uma compreensão profunda das funcionalidades do Mockito e sua aplicação no desenvolvimento de testes unitários robustos em Java

Referências Bibliográficas

KOUSEN, Ken. *Mockito Made Clear: Java Unit Testing with Mocks, Stubs, and Spies*. 1. ed. Raleigh: The Pragmatic Programmers, 2023.

Ir para exercício